

A Proposal for a Code Generator for *Arduino* in the Context of Model-Driven Development for IoT

Eduardo Barbosa (eduardobar7@gmail.com.br[‡]); Paulo Caetano da Silva (Dr.)
(paulocaetano.dasilva@gmail.com)¹

¹ Universidade Salvador – UNIFACS

Summary

This work presents a proposed code generation software specifically for the development of IoT products using the *Arduino platform*. Given the difficulty in writing code manually, and also the existence of obstacles to model-driven development (MDD), such as proprietary tools with insufficient documentation, little exploration and complexity, in addition to the lack of material in Portuguese, this work aims to explore the benefits of MDD through the construction of a modeling environment for IoT products based on the *Arduino platform* for semi-automatic code generation on specific platforms. Future work aims to develop a functional code generator for *Arduino* in accordance with what has been proposed.

Keywords : Code generator, *Arduino*, model-driven development, DDM, IoT.

INTRODUCTION

Model-driven development (MDD) (Stahl & Volter, 2010) is a development approach that emphasizes the use of models as primary development artifacts. In this approach, a system is modeled at a high level of abstraction and semi-automatically converted into less abstract models until the application's source code is generated. This saves time and resources and avoids errors caused by programmers writing code. The framework necessary for MDD development considers two essential elements: models, for representing the system at various levels of abstraction, and transformations, programs responsible for converting the models into code (Brambilla, Cabot, & Wimmer, 2012).

The models built for DDM development need to be written in modeling languages with well-defined syntax and semantics. DDM uses Domain Specific Languages (DSLs) (Fowler, 2011) to build these models, as these languages encapsulate concepts from a specific area, making the models more expressive. *Therefore*, the adoption of DDM requires the definition of DSLs that meet the domain in which the system is embedded. In addition to DSLs, development with DDM also requires the construction of transformations, programs that convert the models into code written for specific platforms.

This work aims to investigate the benefits of using DDM in the development of IoT products. To this end, a code generator for IoT is proposed using a language specific to this domain. The intention of this work is to evaluate the effectiveness of this language in building models independent of the computational platform. In this direction, after evaluating domain-specific

languages available in the literature, we identified that they are still underexplored. Many are complex languages, in addition to being proprietary or having insufficient documentation, which hinders their use. Therefore, we opted to develop our own framework for IoT.

METHOD

The project began with a study phase that involved: understanding the fundamentals of MDD and some concepts related to IoT; the platform selected for building this solution in Java and the NetBeans development environment was also studied ¹. Following this, the implementation of the environment that automates development began.

RESULTS AND DISCUSSION

The DDM solution for IoT proposed in this work comprises a DSL, a modeling environment based on that DSL, and a transformation for generating code in Arduino (Figure 1).



Figure 1 - Proposed tool

The tool's main interface is shown in Figure 2. The interface features a tree-like organization of IoT product *components* on the left and a modeling area on the right, as well as menu options. As components are created for a project, they are added to the tree. Finally, a JSON file containing the model is generated, which is used as input for the *Arduino -specific source code transformer*.

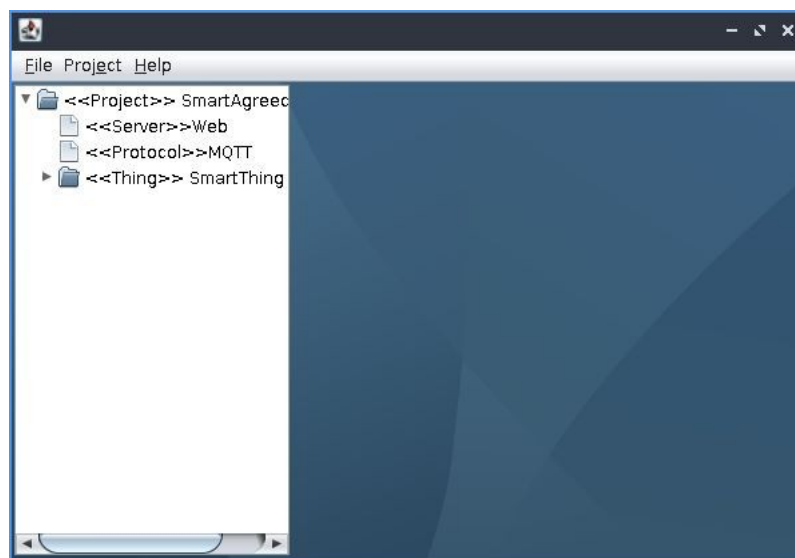


Figure 2 – Tree-shaped application

¹<https://netbeans.apache.org/front/main/index.html>

Next, we will present the architecture of this tool with descriptions of its *components* , details of the tool's implementation, and an example of its use, illustrating how it is configured and executed for transforming code for Arduino.

Figure 3 shows the architecture of the proposed solution according to its metamodel, with the *Components* (*Protocol* , *Server* , and *Thing*) subordinate to the *Project* , which is the configuration file containing the information for the *Components that will be* described below. The *Component Thing* , in turn, has the *Components Controller* and *Component* . The *Project* should be used by a *Transformer that will transform it* into source code for *Arduino* .

- ▣ *Project* – A JSON-formatted configuration file created from a new project. Its purpose, as a configuration file, is to map the IoT *components* (*protocol* , *server*, etc.) for transformation into the *Arduino source code* ;
- ▣ *Thing* – represents an IoT device (e.g., LED, sensor) to be connected to the *Arduino board* ;
- ▣ *Component* – Represents the configuration of a component in *Arduino*; it is a sub-element of *Thing* , inheriting its characteristics. It can be a *Port*, representing a port on the Arduino board, or a *Topic* , representing the message exchange pattern, which can be one of the following: *publish* (sends messages), *subscribe* (receives messages), or *publish/subscribe* (sends and receives messages) (all sub-components of *Thing*).
- ▣ *Controller* – Represents the settings of a controller in *Arduino* , another sub-element of *Thing* . It also has a *Port sub-element* . In addition to its own attributes:
 - *Name* : controller name;
 - *Type* : controller type (ESP32 or *Arduino Uno*).
- ▣ *Protocol* – An architectural element in the same layer as the *Server* and *Thing* elements. It represents the *Arduino* board's protocol settings , which can be *MQTT (MQ Telemetry TransPort)* ² or *TCP (Transmission Control Protocol)* (Fall & Stevens, 2012). It has the following attributes:
 - *Protocol Name* : MQTT or TCP;
 - *User* : username used for authentication;
 - *Password* : password used for authentication.
- ▣ *Server* – Represents the server settings on the *Arduino board* , and has the following attributes:
 - *Server Name* : server name;
 - *Password* : password to access the server;
 - *SSID (Service Set Identifier)*: Wireless network name ;

²<https://mqtt.org/faq/>

- *Port* : represents the server's *port number* .
- ▮ *Port* – Represents a *Port* on the *Arduino board* , and has the following attributes:
 - *Port Name* : *Port name* ;
 - *Port Type* : *Port type* (analog or digital);
 - *Pin Type* : pin type – input and output;
 - *PIN ID* : PIN identifier.
- ▮ *Topic* – Represents the message exchange pattern of a *Component* , and has the following attributes:
 - *Topic Name* : topic name;
 - *Type* : *publish* (sends messages), *subscribe* (receives messages), and *publish/subscribe* (sends and receives messages).
- ▮ *Transformer* - A component of the architecture that transforms the *Project* , with the components configured and described above, into source code for the *Arduino* .

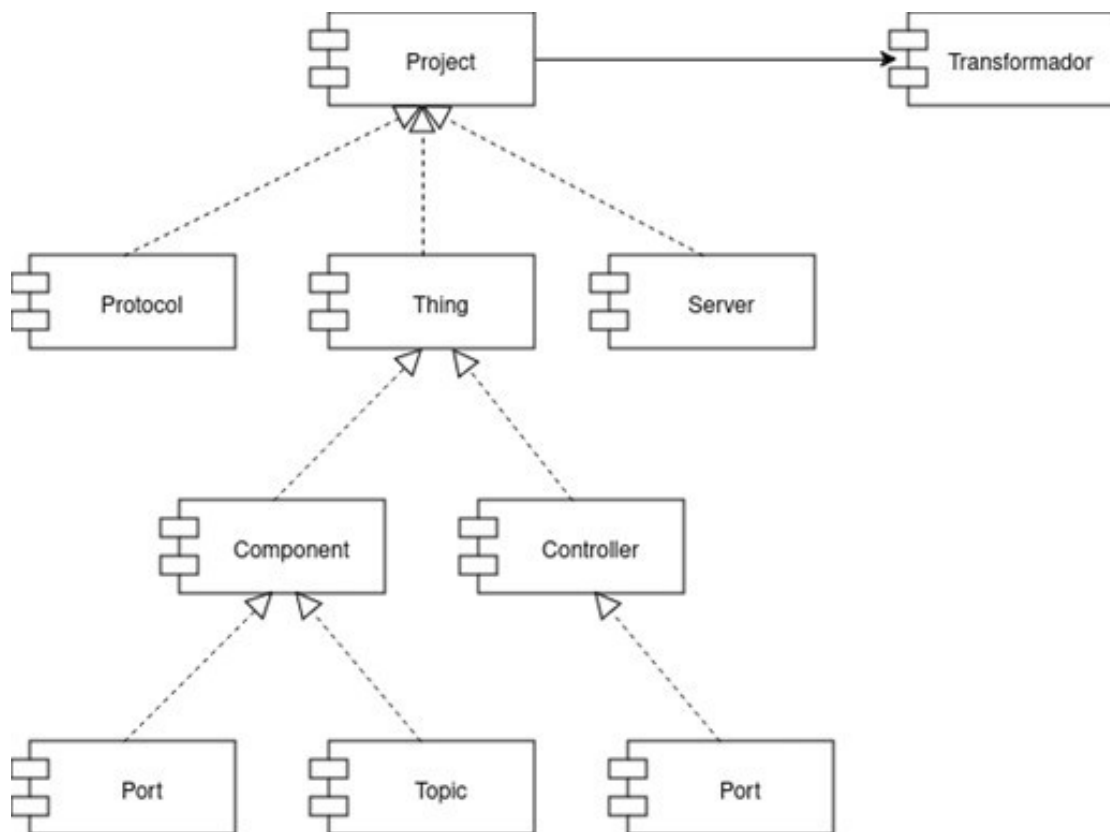


Figure 3 – Proposed architecture

The proposed tool provides a graphical user interface (GUI) that allows the creation of *Arduino projects* using the DDM approach. This proposed use of *DDM* was chosen because it presents specific modeling constructs for IoT applications.

Figure 4 shows the activity diagram of the proposed tool for executing the tool's workflow, as described:

- 1) The user starts the program and creates a new project (New Project);
- 2) The user edits the *Protocol* (Edit Protocol) to enter the *Arduino board's protocol settings* ;
- 3) Next, the user edits the server settings on the *Arduino board* (Edit Server);
- 4) The user creates a new Thing , which will represent an IoT device, e.g., LED, sensor, to be connected to the *Arduino board* ;
- 5) Starting from the creation of the previous step, creating a *Thing* , the user creates a new component (Create new Component) - with a new Port (Create new Port) or a new topic (Create new Topic);
- 6) Finally, the user creates a new controller (Create new Controller) and its new topic (Create new Topic);
- 7) After completing these activities, the user can either transform the settings into an IoT model or save them to avoid losing the configuration and perform the transformation at a future time using the *Transformer* .

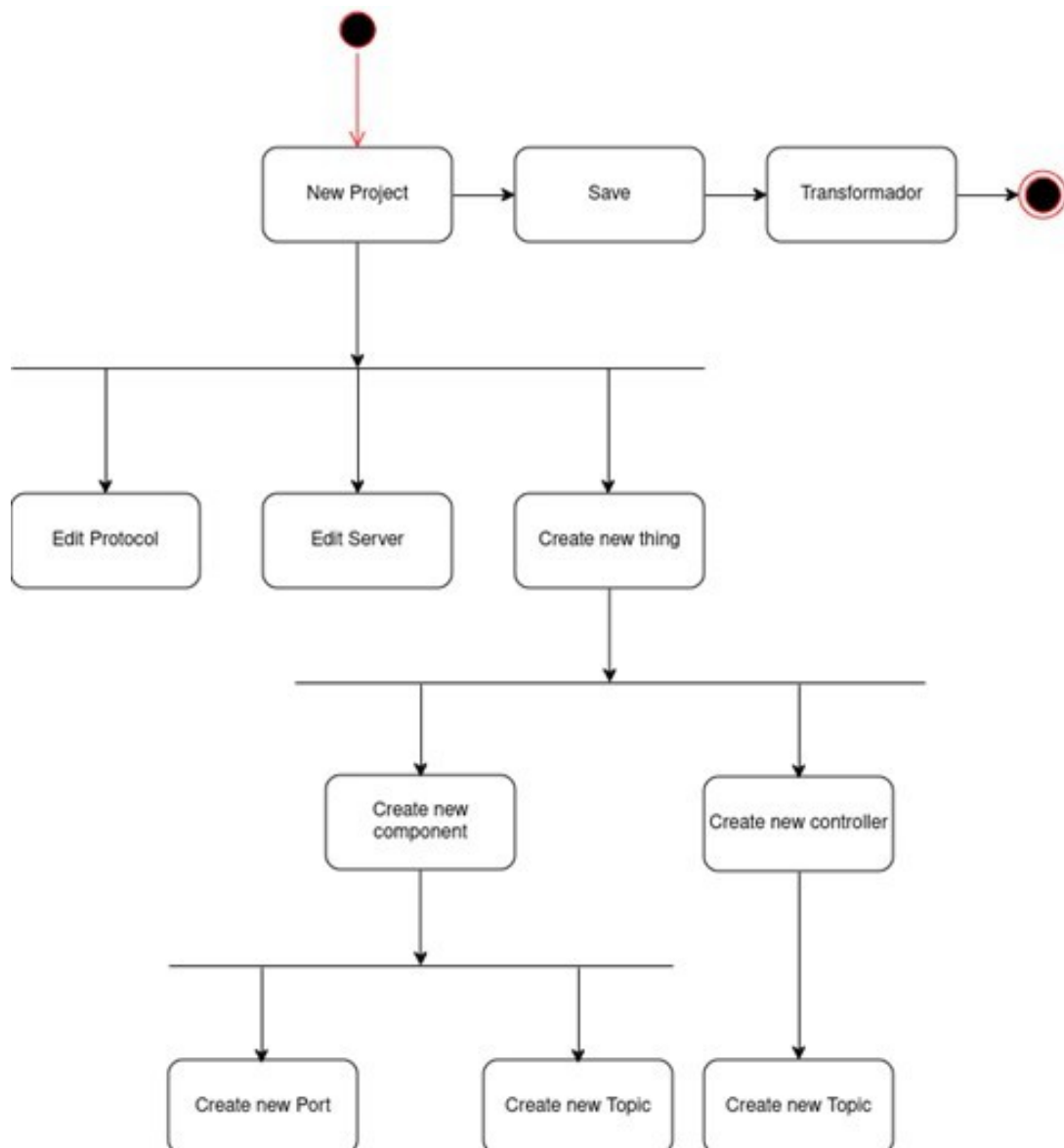


Figure 4 – Activity diagram

On the initial screen, Figure 5, the user accesses the menu in *File* → *New Project* , types the project name (which will also be the name of the folder where the configuration files will be stored), selects the directory path where the folder will be created, and then clicks OK to start the new project. After the new project is created, Figures 6 and 7, the user can define the protocol (e.g., MQTT), Figures 8 and 9, and the web server, Figures 10 and 11, create a new Thing , Figures 12 and 13, and a new component or a new controller (Figure 14). A similar process must be executed to create the controller and port. After finishing the configurations, the user must save them, using the *Save menu* (Figure 15), so that the program saves all the configurations in a *.json file* , Figure 16, in the folder with the same name as the project, in the directory specified in *New Project* . The saved *.json* file can be used by the transformer, which will convert it into *Arduino* -specific source code .

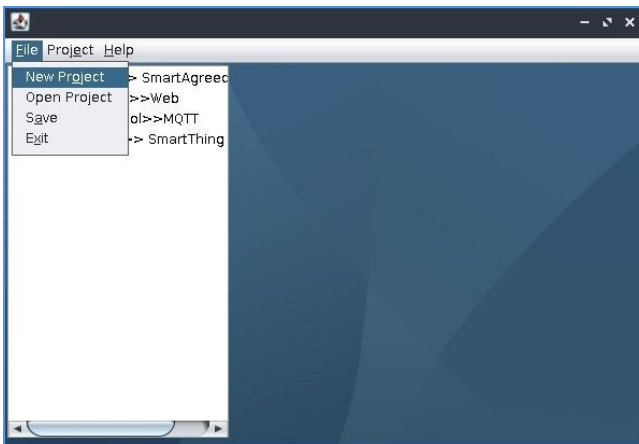


Figure 5 – Initial screen of the tool



Figure 6 – New Project Screen

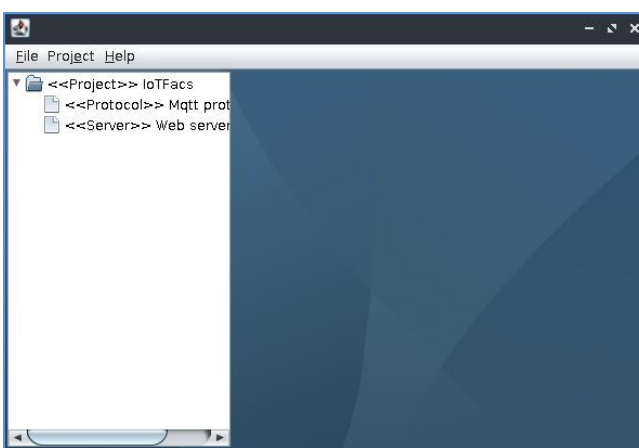


Figure 7 – Screen with information about the new project

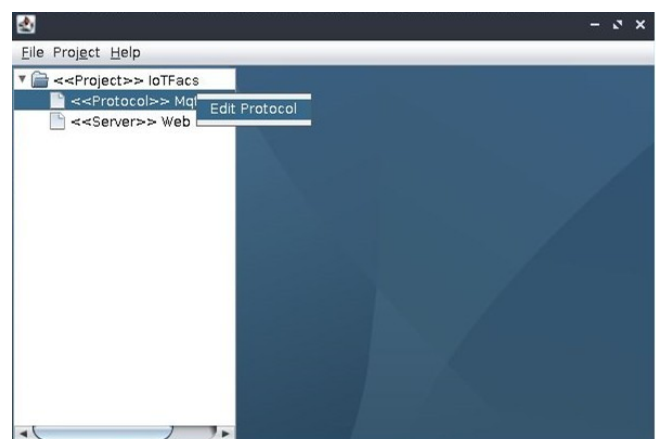


Figure 8 – Creation of the Protocol

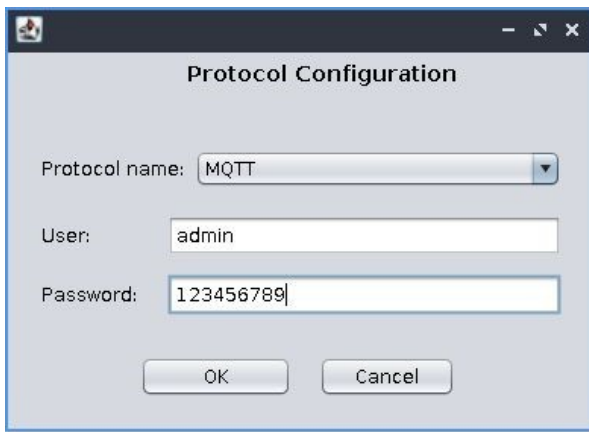


Figure 9 – Protocol definition

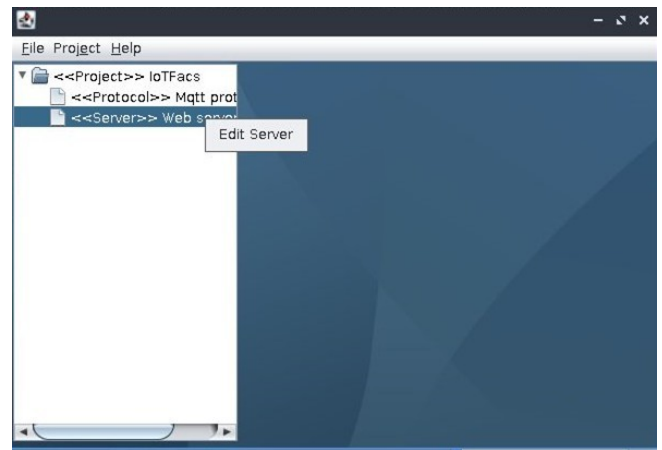


Figure 10 – Server Creation



Figure 11 – Server Definition

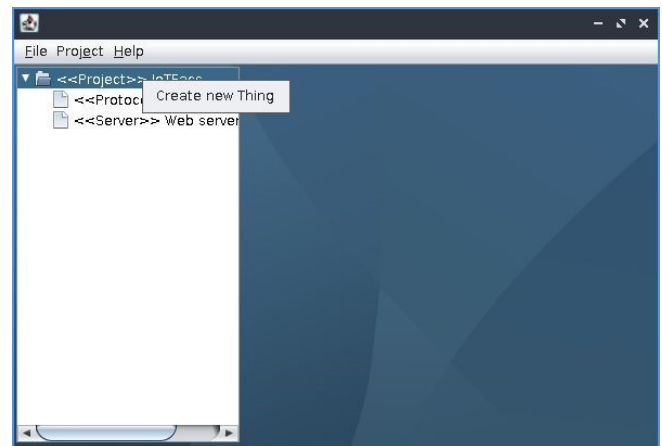


Figure 12 – Creation of the Thing



Figure 13 – Definition of the Thing

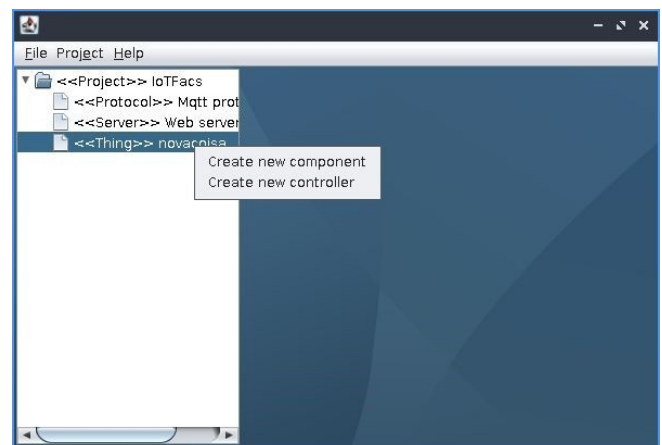


Figure 14 – Component Creation

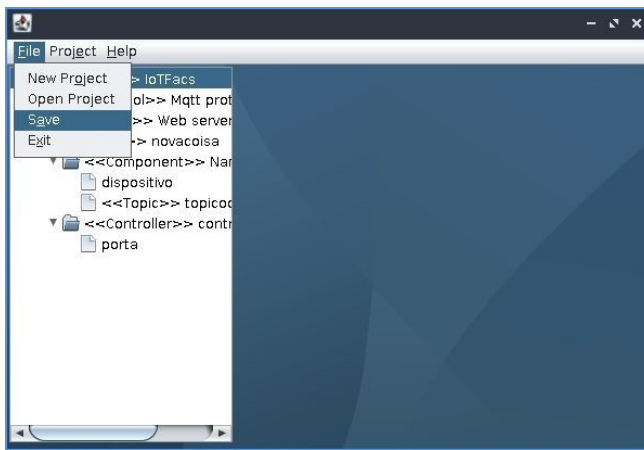


Figure 15 – Saved Project

```

{
  "ProjectNome": "IoTfacs",
  "ProjectFolder": "/home/user/Desktop",
  "Protocol": {
    "nomeProt": "MQTT",
    "userProt": "admin",
    "passwordProt": "123456789"
  },
  "Thing": [
    {
      "nome": "novacoisa",
      "descricao": "coisa nova",
      "controladorList": [
        {
          "Porta": {
            "tipoPinoPorta": "input",
            "idPinoPorta": "12",
            "nomePorta": "dispositivo",
            "analogicoPorta": false
          },
          "nome": "controlador",
          "estado": false
        }
      ],
      "ComponenteList": [
        {
          "Porta": {
            "tipoPinoPorta": "output",
            "idPinoPorta": "11",
            "nomePorta": "dispositivo",
            "analogicoPorta": false
          },
          "nome": "",
          "estado": false
        }
      ],
      "ComponentList": [
        {
          "Topico": {
            "tipoTopico": "publish/subscribe",
            "nomeTopico": "Topicododispositivo"
          },
          "nome": "",
          "estado": false
        }
      ]
    }
  ],
  "Server": {
    "nomeServer": "facs",
    "passwordServer": "123456789",
    "idServer": "unifacs",
    "PortaServer": "1"
  }
}

```

Figure 16 - Example of the JSON configuration file

CONCLUSIONS

This work proposed the development of an environment using the DDM approach for building IoT products. The tool proposed in this work is a development environment for IoT solutions with greater abstraction (emphasis on Arduino hardware), which could provide the creation of source code in a fast and easy way, and could be used in academia as a tool to teach students how to develop IoT solutions. It is expected that this tool can increase the quantity and quality of IoT solutions, due to the facilitation of their development, which could integrate the Internet of Things into society.

REFERENCES

- Brambilla, M., Cabot, J., & Wimmer, M. (2012). Model-Driven Software Engineering in Practice. Morgan & Claypool Publishers.
- Fowler, M. (2011). Domain Specific Languages. Addison Wesley.
- Stahl, T., & Volter, M. (2010). Model-Driven Software Development. Wiley.
- ThingML. (June 2018). ThingML. Source: ThingML: www.ThingML.org
- Thramboulidis, K., & Christoulakis, F. (06 2016). UML4IoT. Source: UML4IoT: <https://sites.google.com/site/uml4iot/>
- Hailpern, B., & Tarr, P. (2006). Model-driven development: The good, the bad, and the ugly.
- Mellor, S. Clark, A., & Futagami, T. (2003). Model-Driven Development
- Mascarenhas, A. (2016). Systematizing the Development of Model-by-Model Transformations in a Model-Driven Approach
- Bowen, J. & Turner, J. (2023). Interactive System Modeling for the Internet of Things
- Pramudianto, F., Rusmita, I., & Jarke, M. (2013). Model Driven Development for Internet of Things Application Prototyping
- Hovsepyan, A., Van Baelen, S., Vanhooft, B., Joosen, W., & Berbers, Y. (2006). Key Research Challenges for Successfully Applying MDD Within Real-Time Embedded Software Development
- Moin, A., Rössler, S., & Günnemann, S. (2018). ThingML+ Augmenting Model-Driven Software Engineering for the Internet of Things with Machine Learning
- Metzger, A. (2005). A Systematic Look at Model Transformations

van Deursen, A., Klint, P., & Visser, J. (2000). Domain-Specific Languages: An Annotated Bibliography

Harrand, N., Fleury, F., Morin, B. & Husa, K.E. (2016). ThingML: A Language and Code Generation Framework for Heterogeneous Targets

Ihirwe, F., Di Ruscio, D., Mazzini, S. & Pierantonio, A. (2021). A domain-specific modeling and analysis environment for complex IoT applications

Høgenesm, J., & Morin, B. (2018). Implementing the MDETools'18 challenge with ThingML

Fall, K.R., & Stevens, W.R. (2012). TCP/IP Illustrated (Vol. 1). Addison-Wesley Professional.

PROMOTION

This work was granted a Scientific Initiation Scholarship by the National Council for Scientific and Technological Development (CNPq).